



TRABAJO FIN DE MÁSTER

Comparación de métodos clásicos y profundos para la detección de música generada por inteligencia artificial

Realizado por
Sergio Álvarez Piñón

Para la obtención del título de
**Máster en Ingeniería del Software: Cloud, Datos y Gestión
TI**

Dirigido por
José María Luna Romera

Convocatoria de Septiembre, curso 2025/26

Agradecimientos

Quiero agradecer a X por...

También quiero agradecer a Y por...

Resumen

Incluya aquí un resumen de los aspectos generales de su trabajo, en español.

Palabras clave: Palabra clave 1, palabra clave 2, ..., palabra clave N

Abstract

This section should contain an English version of the Spanish abstract.

Keywords: Keyword 1, keyword 2, ..., keyword N

Índice general

1. Introducción	1
1.1. Introducción	1
1.2. Estructura de este documento	1
2. Estado del arte	3
2.1. Introducción y delimitación del problema	3
2.2. Conjuntos de datos para detección de música sintética	3
2.3. Enfoques clásicos con características acústicas	4
2.4. Enfoques profundos y representaciones preentrenadas	5
2.5. Segmentación y evaluación a nivel de canción	5
2.6. Generalización, robustez y limitaciones actuales	6
2.7. Conclusiones	7
3. Estudio previo	8
3.1. Introducción	8
3.2. Objetivos	8
3.2.1. Objetivo general	8
3.2.2. Objetivos específicos	8
3.3. Metodología	9
3.3.1. Documentación y revisión bibliográfica	9
3.3.2. Gestión del proyecto y control de versiones	9
3.3.3. Preparación de los datos y reproducibilidad	9
3.3.4. Desarrollo y evaluación experimental	9
3.4. Planificación	10
3.5. Presupuesto	12
4. Descripción de la propuesta	15
4.1. Introducción	15
4.2. Formulación del problema	15
4.3. Selección del conjunto de datos AIME	15
4.4. Auditoría y construcción del subconjunto experimental	16
4.5. Diseño de las particiones	16
4.6. Baseline clásico MFCC + SVM	17
4.7. Protocolo experimental común	18
4.8. Medidas de evaluación	18
4.9. Limitaciones del diseño experimental	19
4.10. Conclusiones	19
5. Validación	20
5.1. Introducción	20
5.2. Entorno y reproducibilidad	20
5.3. Ejecución del baseline	20
5.4. Selección de la configuración	20

5.5. Resultados predictivos y análisis	21
5.6. Tiempos de ejecución registrados	22
5.7. Limitaciones del baseline	23
5.8. Conclusiones	23
6. Conclusiones	24
7. Bibliografía	25

Índice de figuras

3.1. Planificación temporal del TFM entre el 22 de junio y el 15 de septiembre de 2026.	12
5.1. Matriz de confusión del baseline MFCC + SVM en la partición de prueba.	22

1. Introducción

1.1. Introducción

La generación musical mediante inteligencia artificial ha avanzado de forma notable en los últimos años. Los modelos actuales permiten producir piezas musicales completas a partir de descripciones textuales u otras condiciones de entrada, y su uso se ha extendido más allá de entornos técnicos especializados. Este escenario abre posibilidades creativas y productivas, pero también dificulta distinguir con claridad entre música producida por humanos y música generada mediante IA.

La detección de este tipo de contenido se plantea como un problema relevante dentro del análisis de audio. En este TFM, la tarea se formula como una clasificación binaria entre música producida por humanos y música generada mediante IA. La evaluación se centra en el comportamiento de distintos enfoques de clasificación bajo un protocolo experimental común, sin abordar la atribución a un sistema concreto.

El trabajo compara un método clásico, basado en *Mel-frequency cepstral coefficients* (MFCC) y un clasificador *support vector machine* (SVM), con un modelo profundo de audio preentrenado. Esta comparación permite enfrentar una aproximación tradicional, basada en características acústicas resumidas, con una estrategia que aprovecha representaciones aprendidas previamente a partir de grandes colecciones de audio.

Ambos enfoques se evalúan utilizando el mismo conjunto de datos y las mismas particiones. El preprocesamiento se documenta en cada caso y se adapta a los requisitos de entrada de cada modelo sin comprometer la comparabilidad del experimento. De este modo, la evaluación puede registrar tanto medidas de rendimiento predictivo como medidas operacionales.

La inclusión de medidas operacionales responde a la integración posterior del modelo seleccionado en una prueba de concepto web, donde también influyen el tiempo de procesamiento, el tamaño del modelo, el consumo de recursos y la viabilidad de despliegue. Por ello, la selección se basa en el equilibrio entre rendimiento predictivo y coste operacional. La aplicación ofrece una estimación experimental sobre el origen probable del audio analizado, no una certificación de autenticidad.

1.2. Estructura de este documento

El presente documento se organiza en los siguientes capítulos:

- **Introducción:** presenta el contexto del trabajo, la motivación del problema, el alcance de la clasificación binaria y la estructura general de la memoria.

- **Estado del arte:** revisa trabajos y recursos relacionados con la generación musical mediante IA, la detección de música sintética, los conjuntos de datos disponibles y los enfoques de análisis de audio relevantes para este TFM.
- **Estudio previo:** define los objetivos del TFM, la metodología general y los elementos de planificación y presupuesto asociados al desarrollo del proyecto.
- **Descripción de la propuesta:** describe el planteamiento experimental y la arquitectura de la prueba de concepto, incluyendo el conjunto de datos, el preprocesamiento, los modelos comparados y los criterios de selección.
- **Validación:** recoge el protocolo de evaluación, las medidas de rendimiento predictivo y operacional, y la comparación entre el baseline clásico y el modelo profundo seleccionado.
- **Conclusiones:** resume los resultados obtenidos, las limitaciones del trabajo y las posibles líneas de mejora futuras.

2. Estado del arte

2.1. Introducción y delimitación del problema

La detección de música generada mediante inteligencia artificial se ha ido consolidando como un problema propio dentro del análisis de audio. En trabajos recientes, esta tarea aparece bajo la denominación *Synthetic Song Detection* (SSD), entendida como la clasificación entre canciones producidas por humanos y canciones generadas de extremo a extremo mediante sistemas de IA [17]. Esta formulación es la que se adopta en este TFM: una tarea binaria centrada en distinguir música producida por humanos frente a música generada mediante IA.

Esta delimitación es importante porque existen tareas cercanas que conviene distinguir. La *Singing Voice Deepfake Detection* se centra en detectar voces cantadas sintéticas o convertidas, a menudo dentro de mezclas donde el acompañamiento puede proceder de música real [19, 18]. La detección de canciones sintéticas de extremo a extremo, en cambio, considera que pueden ser artificiales la voz, la instrumentación, la letra, el arreglo y la producción global. También quedan fuera del alcance principal la detección de voz hablada sintética, la detección de audio sintético general, la detección de letras generadas mediante IA y la atribución del modelo generador.

El interés de SSD va más allá de obtener una decisión automática sobre un archivo de audio. La música es una señal con estructura temporal, patrones tímbricos, organización rítmica y relaciones de larga duración. Por ello, los detectores pueden apoyarse tanto en indicios locales de la señal como en propiedades musicales más amplias. Esta combinación explica que el campo incluya desde enfoques clásicos basados en características acústicas hasta modelos profundos que intentan aprovechar representaciones aprendidas y relaciones entre fragmentos.

2.2. Conjuntos de datos para detección de música sintética

Uno de los recursos más directamente orientados a SSD es SONICS, que plantea explícitamente la detección de canciones sintéticas de extremo a extremo. Sus autores subrayan que muchos conjuntos de datos previos se centraban en voz cantada sintética, mientras que las canciones generadas actualmente pueden ser artificiales en todos sus componentes [17]. Esta diferencia justifica la necesidad de corpus específicos para canciones completas y de protocolos que tengan en cuenta dependencias temporales más largas que las de un fragmento aislado.

FakeMusicCaps aborda un problema próximo desde otra perspectiva. El conjunto de datos se construye a partir de MusicCaps, regenerando pares música-texto mediante varios modelos *text-to-music*. Además de la detección binaria, el trabajo estudia la

atribución del modelo generador [3]. En este TFM se toma como ejemplo de corpus específico para música generada por IA y de cómo la detección puede verse afectada por el origen de los generadores incluidos.

AIME ocupa una posición distinta. La tarjeta oficial del conjunto de datos lo presenta como *AI Music Evaluation Dataset*, con pistas generadas por distintos modelos y pistas procedentes de MTG-Jamendo, junto con metadatos como el modelo de origen y una descripción basada en etiquetas [5]. Su finalidad original está vinculada a la evaluación de modelos de generación musical y preferencias humanas, más que a un conjunto de referencia estándar para detección [8]. Por tanto, en este TFM AIME se reutiliza como corpus documentado y auditado para una tarea binaria, con las cautelas metodológicas necesarias.

La reutilización de AIME para detección cuenta al menos con un antecedente verificado: *Fusion Segment Transformer*, identificado como preprint de 2026, evalúa su propuesta en SONICS y AIME [12]. Esta evidencia es todavía limitada, aunque muestra que el corpus puede emplearse en trabajos de detección cuando se explicita el protocolo experimental. La distinción es relevante para evitar presentar el conjunto de datos como si hubiera nacido con una partición o una metodología oficial para detección.

2.3. Enfoques clásicos con características acústicas

Los enfoques clásicos de clasificación de audio suelen partir de características acústicas calculadas manualmente. Estas representaciones resumen propiedades de la señal antes de aplicar un clasificador, lo que permite construir sistemas relativamente simples, reproducibles y de bajo coste computacional. En este contexto, los MFCC representan información espectral de corto plazo mediante vectores compactos.

En detección de audio sintético, los MFCC aparecen como una representación transferible desde tareas cercanas, especialmente en sistemas de anti-*spoofing* de voz. El precedente aplicado citado en este capítulo es Novoselov et al., que emplean MFCC dentro del reto ASVspoof 2015 y comparan clasificadores clásicos con enfoques más complejos [15]. Aunque este dominio no coincide con SSD, sirve como antecedente metodológico para entender por qué MFCC puede funcionar como una línea base razonable en problemas de detección acústica.

La SVM es un clasificador clásico para tareas supervisadas, especialmente adecuado cuando se trabaja con vectores de características fijas. Su formulación de margen máximo y el uso de kernels permiten construir fronteras de decisión no lineales sin requerir arquitecturas profundas [4]. En este TFM, la combinación MFCC + SVM debe entenderse como baseline clásico: una referencia sencilla y controlable frente a la que comparar modelos más expresivos, más que como el enfoque más avanzado del estado del arte actual.

La principal limitación de este tipo de aproximación es que resume la señal en descriptores compactos. Esto puede ser suficiente para capturar ciertos patrones espectrales, pero deja menos margen para modelar relaciones temporales largas, estructura musical o dependencias entre secciones de una canción. Por esa razón, la comparación

con un modelo profundo preentrenado resulta útil para evaluar si las representaciones aprendidas aportan ventajas bajo un protocolo común.

2.4. Enfoques profundos y representaciones preentrenadas

El desarrollo reciente de SSD se ha desplazado hacia modelos profundos capaces de aprender representaciones más ricas a partir del audio. SONICS propone SpecTT-Tra, una arquitectura orientada a capturar dependencias temporales largas con menor coste que algunos modelos convolucionales o transformadores de referencia [17]. Esta línea pone de manifiesto que la detección de canciones completas depende tanto de indicios locales como de la forma en que la música se organiza a lo largo del tiempo.

Segment Transformer sigue una dirección complementaria: divide el audio en segmentos, extrae información de cada uno mediante modelos preentrenados o auto-supervisados y aprende relaciones entre fragmentos para mejorar la detección a escala de canción [11]. La propuesta muestra la utilidad de separar el análisis local de los fragmentos y el modelado estructural posterior, especialmente cuando el audio completo es demasiado largo para tratarse de una sola vez.

El preprint *Fusion Segment Transformer* amplía esta idea mediante una fusión guiada de información de contenido y estructura. Su interés para este TFM es doble: por un lado, confirma la relevancia de los enfoques por segmentos; por otro, proporciona una evidencia directa de evaluación sobre AIME dentro de un trabajo de detección [12]. Esta línea se utiliza aquí como referencia metodológica, sin asumir que el modelo profundo del TFM deba pertenecer a esa arquitectura concreta.

Los encoders de audio preentrenados forman una familia metodológica especialmente relevante para comparar con un baseline clásico. Estos modelos permiten extraer representaciones aprendidas en tareas o corpus amplios y reutilizarlas en una tarea de clasificación posterior. En el contexto de SSD, su interés está en aprovechar información más rica que la contenida en descriptores manuales, aunque la elección concreta del encoder debe depender de criterios experimentales y operacionales antes que de una decisión tomada solo a partir de la literatura [11, 12].

2.5. Segmentación y evaluación a nivel de canción

La segmentación aparece de forma natural en detección musical porque las canciones pueden tener duraciones superiores a las que muchos modelos procesan directamente. Trabajar con fragmentos permite adaptar el audio a modelos con entradas de duración fija, acotar el coste de cada inferencia y obtener varias observaciones de una misma pieza. La decisión a nivel de canción requiere, además, una estrategia de agregación adecuada.

En tareas de clasificación musical, la forma de dividir el audio y agregar predicciones puede afectar a las medidas de rendimiento. Nasrullah y Zhao muestran, en clasificación de artistas, que la longitud de los fragmentos, el esquema de particionado y la agregación a nivel de canción influyen en la evaluación [14]. Aunque la tarea es distinta, la lección metodológica es transferible: el protocolo de segmentación debe documentarse y evitar que información relacionada aparezca simultáneamente en entrenamiento y prueba.

Los trabajos específicos de SSD también refuerzan esta idea. SONICS destaca la importancia de las dependencias de larga duración, mientras que Segment Transformer modela explícitamente relaciones entre segmentos [17, 11]. Esto sugiere que un detector puede operar sobre fragmentos y, al mismo tiempo, necesitar una estrategia de agregación si se desea emitir una decisión sobre una canción completa. Los trabajos revisados no establecen una duración universal para los fragmentos, por lo que esa elección debe justificarse dentro del protocolo experimental concreto.

2.6. Generalización, robustez y limitaciones actuales

Una dificultad recurrente en SSD es la generalización fuera del escenario de entrenamiento. Afchar et al. muestran que obtener una medida alta en una prueba cerrada resulta insuficiente para considerar resuelto el problema, ya que la robustez frente a manipulaciones del audio y la generalización a generadores no vistos siguen siendo cuestiones críticas [1]. Esta advertencia es especialmente importante cuando los detectores pueden aprender atajos acústicos asociados a un conjunto de datos, a un códec o a un proceso de producción concreto.

Los cambios de dominio también aparecen en trabajos adyacentes de SVDD. SingFake identifica dificultades con cantantes no vistos, códecs de comunicación, idiomas y contextos musicales [19]. CtrSVDD, por su parte, destaca que la selección de características y la separación entre métodos de falsificación vistos y no vistos afectan al comportamiento del detector [18]. Aunque pertenecen a una tarea distinta, estos resultados ofrecen evidencia transferible sobre la fragilidad de los detectores cuando cambian las condiciones de evaluación.

Los preprints recientes de 2026 insisten en esa misma dirección desde la detección de música generada mediante IA. MusicDET plantea explícitamente la detección ante generadores no vistos mediante un enfoque *zero-shot* [9]. Broadcast Monitoring analiza un escenario donde la música aparece en fragmentos cortos o enmascarada por habla, y muestra que las condiciones de emisión pueden degradar modelos entrenados para contextos más limpios [13]. ArtifactNet, también como preprint, centra la atención en la robustez frente a códecs y en detectores ligeros basados en residuos forenses [16].

En conjunto, estas aportaciones muestran que las medidas obtenidas en un corpus cerrado deben interpretarse con cautela. La duración de los fragmentos, el idioma, el artista, el género, la frecuencia de muestreo, la compresión y los artefactos de producción pueden influir en la decisión del modelo. Por tanto, una comparación experimental rigurosa debe distinguir entre rendimiento dentro del conjunto de datos evaluado y capacidad de generalización a condiciones externas.

2.7. Conclusiones

Synthetic Song Detection es una tarea reciente y todavía en desarrollo. Aunque ya existen conjuntos de datos y arquitecturas específicas para música generada de extremo a extremo, todavía falta un enfoque consolidado que pueda considerarse solución general. Los resultados en escenarios cerrados son útiles para comparar modelos, pero ofrecen garantías limitadas ante nuevos generadores, transformaciones de audio o cambios de dominio.

La revisión realizada permite justificar la comparación entre un baseline clásico MFCC + SVM y un modelo profundo preentrenado bajo un protocolo común. El baseline aporta una referencia reproducible y de bajo coste, mientras que los modelos profundos y las representaciones preentrenadas ofrecen una vía para capturar información más rica de la señal musical. Esta comparación debe complementarse con medidas operacionales, especialmente si el modelo seleccionado se integra en una prueba de concepto web.

Por último, AIME debe presentarse con precisión: es un corpus documentado procedente de la evaluación de modelos de generación musical, reutilizado aquí para detección binaria tras una auditoría propia. Esta formulación evita tratarlo como conjunto de referencia estándar para SSD y permite situar correctamente las limitaciones del experimento.

3. Estudio previo

3.1. Introducción

Este capítulo presenta los objetivos del TFM, la metodología de trabajo, la planificación temporal y el presupuesto estimado del proyecto. Estos elementos establecen el marco de organización y seguimiento aplicado durante su desarrollo.

3.2. Objetivos

3.2.1. Objetivo general

Comparar un enfoque clásico basado en MFCC y SVM con un modelo profundo preentrenado para la detección de música generada mediante inteligencia artificial, evaluando ambos mediante medidas predictivas y operacionales bajo un protocolo experimental común.

3.2.2. Objetivos específicos

1. Revisar el estado del arte relacionado con la detección de música generada mediante IA.
2. Auditar y preparar AIME para su uso experimental.
3. Definir un protocolo reproducible de particionado, preprocesamiento y evaluación.
4. Implementar y evaluar el baseline MFCC + SVM.
5. Seleccionar, adaptar y evaluar un modelo profundo preentrenado.
6. Comparar ambos enfoques y seleccionar el modelo más adecuado para la prueba de concepto, considerando conjuntamente su rendimiento predictivo y su viabilidad operacional.
7. Desarrollar y desplegar una prueba de concepto web que integre el modelo seleccionado.
8. Documentar decisiones, experimentos y artefactos para garantizar la reproducibilidad.

3.3. Metodología

El proyecto sigue un desarrollo iterativo e incremental organizado mediante fases, issues y ciclos de trabajo breves. Esta forma de trabajo permite dividir el TFM en tareas manejables, revisar los resultados de cada etapa y mantener trazabilidad entre decisiones, experimentos y artefactos generados.

3.3.1. Documentación y revisión bibliográfica

La memoria se redacta en LaTeX mediante Overleaf, manteniendo una estructura de capítulos coherente con los requisitos del TFM. La revisión bibliográfica se organiza de forma estructurada y trazable, de modo que las fuentes empleadas puedan relacionarse con las decisiones metodológicas y con el estado del arte descrito en la memoria.

3.3.2. Gestión del proyecto y control de versiones

El desarrollo se gestiona con Git y GitHub, utilizando ramas e issues para separar tareas y registrar cada bloque de trabajo. Las decisiones relevantes, los experimentos ejecutados y los artefactos producidos se documentan en el repositorio para facilitar su revisión posterior.

3.3.3. Preparación de los datos y reproducibilidad

La preparación experimental parte de la selección y auditoría de AIME como conjunto de datos principal. Antes del procesamiento de audio se generan particiones reproducibles, separando entrenamiento, validación y prueba. Esta separación permite controlar fugas de información y mantener el mismo conjunto de datos para evaluar los dos enfoques comparados.

3.3.4. Desarrollo y evaluación experimental

Los modelos se evalúan bajo el mismo protocolo experimental. La partición de validación se emplea para seleccionar las configuraciones, mientras que la partición de prueba se reserva para la evaluación final. Durante el proceso se registran medidas predictivas y operacionales, y se incorporan pruebas automatizadas sobre los componentes críticos para comprobar que el particionado, el preprocesamiento y la evaluación respetan las condiciones definidas y no introducen fugas de información.

3.4. Planificación

La planificación del TFM se ha definido para una dedicación total de 300 horas, correspondiente a los 12 ECTS del trabajo, durante el periodo comprendido entre el 22 de junio y el 15 de septiembre de 2026. La organización sigue un enfoque iterativo e incremental: GitHub Projects y el tablero Kanban estructuran el trabajo, las tareas se registran mediante issues, los cambios se desarrollan en ramas, los commits documentan la evolución del repositorio y las pull requests facilitan la revisión e integración de cada incremento. La documentación se actualiza de forma progresiva conforme se consolidan los avances experimentales y de desarrollo.

La Tabla [3.1](#) resume las fases previstas, su periodo, la dedicación asignada y el resultado verificable asociado a cada una de ellas.

Fase	Periodo	Horas	Resultado o entregable
Alcance, planificación y entorno	22-30 de junio	20 h	Alcance y objetivos definidos, viabilidad estudiada y entorno de trabajo configurado
Auditoría de AIME y preparación de particiones	1-11 de julio	24 h	Conjunto de datos auditado y particiones reproducibles preparadas
Revisión bibliográfica y estado del arte	22 de junio-16 de julio	26 h	Estado del arte actualizado y bibliografía relevante organizada
Baseline MFCC + SVM	12-14 de julio	22 h	Baseline implementado y evaluado mediante el protocolo experimental definido
Documentación del primer incremento	15-20 de julio	18 h	Metodología, dataset, particiones y baseline incorporados a la documentación
Selección y adaptación del modelo profundo	21 de julio-5 de agosto	52 h	Modelo profundo seleccionado, adaptado y evaluado
Comparación experimental y selección del modelo	6-12 de agosto	28 h	Comparación predictiva y operacional completada y modelo de despliegue seleccionado
Documentación del segundo incremento	13-16 de agosto	14 h	Modelo profundo, comparación, resultados y limitaciones documentados
Diseño e implementación de la aplicación web	17 de agosto-2 de septiembre	48 h	Aplicación web funcional con frontend, backend e integración del modelo seleccionado
Pruebas, despliegue y documentación de la aplicación	3-7 de septiembre	18 h	Evidencias de prueba, despliegue académico y documentación de la aplicación
Revisión final de memoria y artefactos	8-13 de septiembre	20 h	Memoria revisada y artefactos técnicos organizados para la entrega
Preparación de la presentación	14-15 de septiembre	10 h	Presentación y demostración preparadas
Total	22 de junio-15 de septiembre	300 h	TFM preparado para su entrega

Tabla 3.1: Planificación temporal y distribución de horas del TFM.

La Figura 3.1 representa visualmente el cronograma del trabajo y permite observar el solapamiento parcial entre la revisión bibliográfica, la preparación del dataset y los primeros incrementos experimentales.

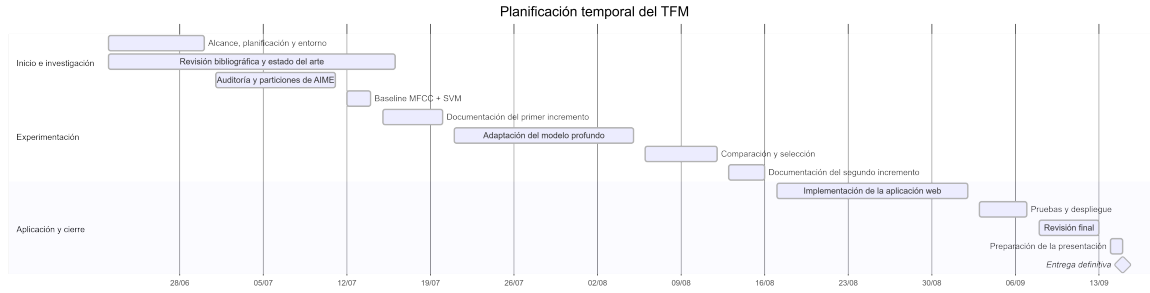


Figura 3.1: Planificación temporal del TFM entre el 22 de junio y el 15 de septiembre de 2026.

Las principales dependencias entre las fases son las siguientes:

- La auditoría de AIME y la generación de las particiones preceden a la ejecución de los experimentos.
- El baseline y el modelo profundo deben evaluarse mediante el mismo protocolo antes de realizar la comparación.
- La selección del modelo para la aplicación depende de los resultados predictivos y operacionales de ambos enfoques.
- La implementación web requiere el artefacto y el preprocesamiento del modelo seleccionado, mientras que las pruebas y el despliegue preceden al cierre del proyecto.

3.5. Presupuesto

El presupuesto estima el coste económico asociado al desarrollo del TFM durante las 300 horas de dedicación planificada. Su finalidad es valorar el esfuerzo y los recursos necesarios para el trabajo académico, por lo que distingue entre costes imputados, como la dedicación, la amortización del hardware, la electricidad y la conexión a Internet, y costes directos de herramientas, servicios y datos. Se interpreta como una estimación presupuestaria del proyecto académico.

Recursos humanos

Para valorar la dedicación se toma como referencia el coste laboral medio por hora efectiva publicado por el Instituto Nacional de Estadística para el primer trimestre de 2026, de 24,88 €/h [10]. Aplicado a las 300 horas planificadas, el coste imputado de los recursos humanos asciende a 7.464,00 €.

$$300 \text{ h} \times 24,88 \text{ €/h} = 7.464,00 \text{ €}$$

Recursos materiales, suministros y comunicaciones

La amortización del hardware se calcula a partir de precios estimados representativos y de una vida útil de cuatro años. Esta vida útil equivale a una amortización lineal anual del 25 %, porcentaje que coincide con el coeficiente lineal máximo indicado por la Agencia Tributaria para equipos destinados a procesos de información [2]. El coste imputado al TFM se obtiene aplicando el periodo de 86 días mediante la expresión:

$$\text{coste amortizado} = \text{precio estimado} \times \frac{86}{4 \times 365}.$$

Recurso	Precio estimado	Vida útil	Periodo imputado	Coste amortizado
Portátil	900,00 €	4 años	86 días	53,01 €
Monitor	110,00 €	4 años	86 días	6,48 €
Periféricos	45,00 €	4 años	86 días	2,65 €
Total				62,14 €

Tabla 3.2: Coste imputado por amortización del hardware durante el periodo del TFM.

El coste eléctrico se estima con una potencia media de 0,12 kW para el conjunto formado por el portátil, el monitor y los periféricos. Se trata de un supuesto de consumo medio, no de una medición directa. Como precio de referencia se utiliza 0,2669 €/kWh, correspondiente a Eurostat para España durante el segundo semestre de 2025, con impuestos incluidos y banda doméstica de 2.500 a 4.999 kWh [6]:

$$0,12 \text{ kW} \times 300 \text{ h} \times 0,2669 \text{ €/kWh} = 9,61 \text{ €}$$

La conexión a Internet se valora mediante una cuota doméstica representativa de 30,00 € mensuales. Para el periodo imputado de 86 días, equivalente a 2,827 meses, el cálculo se realiza de forma proporcional sobre el año completo:

$$30,00 \times \frac{86}{365} \times 12 = 84,82 \text{ €}$$

Software, herramientas, servicios y datos

Los servicios empleados o previstos para la gestión, documentación, publicación y despliegue están disponibles mediante planes gratuitos o acceso abierto, por lo que su coste directo imputado al proyecto es de 0,00 €. La Tabla 3.3 agrupa los recursos considerados.

Recurso	Modalidad	Coste
GitHub y GitHub Projects	Plan gratuito	0,00 €
Clockify	Plan gratuito	0,00 €
Overleaf	Plan gratuito	0,00 €
Hugging Face Hub y Hugging Face Spaces	Plan gratuito	0,00 €
Vercel	Plan gratuito	0,00 €
Dataset AIME	Acceso gratuito; licencias según procedencia	0,00 €
Total de software, servicios y datos		0,00 €

Tabla 3.3: Software, herramientas, servicios y datos considerados en el presupuesto.

Resumen final

La Tabla 3.4 resume el presupuesto estimado del TFM. El coste total imputado asciende a 7.620,57 €.

Categoría	Coste
Recursos humanos	7.464,00 €
Amortización de hardware	62,14 €
Electricidad	9,61 €
Internet	84,82 €
Software, servicios y datos	0,00 €
Total estimado	7.620,57 €

Tabla 3.4: Resumen final del presupuesto estimado del TFM.

4. Descripción de la propuesta

4.1. Introducción

Este capítulo presenta el diseño experimental del TFM. A partir de AIME se construyen el subconjunto de trabajo y sus particiones, y se define el baseline MFCC + SVM dentro del protocolo común de comparación con un modelo profundo preentrenado.

4.2. Formulación del problema

El problema se formula como una clasificación binaria entre música producida por humanos y música generada mediante inteligencia artificial. Cada ejemplo del subconjunto experimental recibe una etiqueta: la clase 0 corresponde a audio humano procedente de MTG-Jamendo, mientras que la clase 1 corresponde a audio generado mediante IA.

Los dos enfoques comparados trabajan sobre el mismo conjunto de ejemplos y las mismas particiones. Esta decisión permite evaluar las diferencias entre modelos sin introducir variaciones en la composición de las particiones de entrenamiento, validación y prueba. El objetivo del experimento es estimar la capacidad de discriminación entre ambas clases, sin atribuir una muestra a un generador concreto ni interpretar la salida como una medida calibrada sobre el origen del audio.

4.3. Selección del conjunto de datos AIME

AIME se utiliza como dataset principal del TFM. La tarjeta oficial lo presenta como *AI Music Evaluation Dataset* e incluye pistas generadas por distintos modelos, pistas procedentes de MTG-Jamendo y metadatos sobre su procedencia y descripción [5]. Estas características permiten construir un subconjunto equilibrado y definir un protocolo propio para la clasificación binaria.

Las condiciones de uso dependen de la procedencia del contenido: las 6 000 pistas generadas se distribuyen bajo licencia CC BY 4.0; las 500 pistas procedentes de MTG-Jamendo conservan la licencia individual de cada pista; y sus etiquetas descriptivas se distribuyen bajo licencia CC BY-NC-SA 4.0 [5].

La elección de AIME se justifica por tres motivos principales:

- disponibilidad documentada de ejemplos humanos y generados;
- diversidad de generadores dentro de la clase IA;
- existencia de metadatos útiles para el etiquetado y el particionado.

Esta elección hace necesario definir un subconjunto experimental propio y un particionado adaptado a la tarea binaria del TFM.

4.4. Auditoría y construcción del subconjunto experimental

La auditoría técnica de AIME abarca 6 500 filas. En la estructura observada aparecen 500 ejemplos humanos procedentes de MTG-Jamendo y 6 000 ejemplos generados mediante IA. Estos últimos están repartidos entre doce generadores, con 500 muestras por cada fuente generativa.

En AIME, el campo *description* contiene tres etiquetas separadas por comas que describen cada muestra [5]. En los metadatos auditados se identificaron 500 combinaciones distintas; por ejemplo, *ambient, blues, piano*. Cada combinación aparece en trece filas: una correspondiente a una muestra humana de MTG-Jamendo y doce correspondientes a muestras generadas, una por cada modelo incluido en el conjunto de datos.

En este TFM, las muestras que presentan la misma combinación de etiquetas se consideran parte de un mismo grupo semántico. Comparten la misma información de etiquetas, aunque los audios proceden de fuentes diferentes. Esta estructura permite vincular una muestra humana con las doce generaciones asociadas y utilizar el grupo como unidad para construir el subconjunto experimental.

Para cada uno de los 500 grupos se conserva la muestra humana y se selecciona una de las doce muestras generadas. El resultado contiene dos ejemplos por grupo y un total de 1 000 muestras: 500 humanas y 500 generadas. La selección de la clase generada se distribuye de forma prácticamente uniforme entre los doce modelos para mantener diversidad dentro de la clase IA sin convertir el problema en una tarea de atribución.

La regla de etiquetado binario se define a partir del campo *model*. Las muestras cuyo valor es MTG-Jamendo se etiquetan como clase humana, mientras que las asociadas al resto de modelos se consideran generadas mediante IA. Esta regla es coherente con la estructura observada y permite formular el experimento como una clasificación binaria.

4.5. Diseño de las particiones

Las particiones se construyen antes de extraer fragmentos temporales o características. El manifiesto `data/aime_splits.csv` contiene 1 000 filas y fija tres particiones: entrenamiento, validación y prueba. Cada partición mantiene balance entre clases, como se muestra en la Tabla 4.1.

Partición	Total	Humanos	Generados
Entrenamiento	700	350	350
Validación	150	75	75
Prueba	150	75	75

Tabla 4.1: Distribución de ejemplos por partición y clase en el manifiesto experimental.

El particionado utiliza como unidad de agrupación cada combinación de etiquetas. La muestra humana y la muestra generada seleccionada para un mismo grupo se asignan siempre a la misma partición. De esta forma, una combinación idéntica no puede aparecer simultáneamente en entrenamiento, validación o prueba, lo que controla una posible fuga semántica.

Las particiones se generan con la semilla 42 mediante una estrategia de particionado equilibrado por generador y agrupación semántica. El manifiesto contiene 350 grupos en entrenamiento, 75 en validación y 75 en prueba. Las pruebas automáticas verifican el número total de filas, el balance de clases, la permanencia de cada grupo en una única partición, la reproducibilidad del manifiesto generado y la presencia de todos los generadores en cada partición.

En el subconjunto experimental están representados los doce generadores disponibles. La distribución es prácticamente uniforme: ocho generadores aportan 42 ejemplos, mientras que AudioLDM 2 Large, MusicGen Medium, Riffusion y Suno v3.5 aportan 41 ejemplos.

4.6. Baseline clásico MFCC + SVM

El baseline clásico se define como un pipeline a nivel de fragmento. Cada señal se representa mediante muestras en coma flotante de 32 bits. Cuando el archivo contiene varios canales, estos se promedian para obtener una única señal monofónica, de modo que todos los ejemplos compartan el mismo formato de entrada.

A continuación se selecciona un fragmento consecutivo de diez segundos. Cuando el audio supera esa duración, se comparan todas las ventanas posibles de diez segundos mediante la suma de los cuadrados de sus muestras y se conserva la ventana con mayor energía. Esta elección prioriza el tramo con mayor actividad de señal dentro del archivo. En audios más cortos, la señal se completa con ceros hasta alcanzar la duración objetivo. El fragmento resultante se remuestrea a 16 kHz y queda representado por 160 000 muestras.

La representación acústica se basa en 20 MFCC calculados para cada fragmento. Cada coeficiente varía a lo largo del tiempo, por lo que se resume mediante su media y su desviación típica. Los 20 valores medios y los 20 valores de desviación producen un vector final de 40 características, que se almacena para poder reutilizarlo durante el entrenamiento y la evaluación.

La clasificación se realiza con un pipeline formado por *StandardScaler* y una SVM con kernel RBF [4]. *StandardScaler* centra cada característica y la escala según

la desviación calculada en entrenamiento; sus parámetros se ajustan únicamente con la partición de entrenamiento para evitar fugas de información. La SVM utiliza el kernel RBF para modelar una frontera de decisión no lineal sobre el espacio de características.

El clasificador emplea *decision_function*, que devuelve un *score* continuo asociado al margen de decisión. Este *score* no es una probabilidad: su signo determina el lado de la frontera en el que se sitúa la muestra y su magnitud refleja la separación respecto a dicha frontera. Los *scores* se emplean para calcular ROC-AUC.

La selección de hiperparámetros contempla 16 configuraciones, resultantes de combinar $C \in \{0,1; 1; 10; 100\}$ y $\gamma \in \{scale; 0,001; 0,01; 0,1\}$. Cada configuración se entrena con la partición de entrenamiento y se evalúa en la partición de validación. El criterio principal es *balanced accuracy*; en caso de empate, se utiliza F1 de la clase IA como segundo criterio. La partición de prueba se reserva para la evaluación final.

4.7. Protocolo experimental común

El protocolo experimental común fija los elementos que deben compartir el baseline MFCC + SVM y el modelo profundo preentrenado. Ambos enfoques utilizan el mismo subconjunto, las mismas etiquetas binarias y las mismas particiones de entrenamiento, validación y prueba. La validación se reserva para seleccionar configuraciones, mientras que la partición de prueba se mantiene para la evaluación final.

El protocolo permite adaptar el preprocesamiento a los requisitos de cada enfoque, siempre que se conserven las mismas muestras, etiquetas y particiones, y que las transformaciones aplicadas queden documentadas.

La configuración de entrada de cada enfoque se documenta junto con su evaluación. El protocolo también contempla la conservación de información suficiente para reproducir la comparación y revisar posibles errores de particionado, preprocesamiento o evaluación.

4.8. Medidas de evaluación

La evaluación predictiva emplea la clase IA como clase positiva. *Balanced accuracy* promedia el recall de ambas clases y otorga el mismo peso a la clase humana y a la clase IA, por lo que se utiliza como criterio principal de selección. En caso de empate se emplea F1 de la clase IA como criterio secundario, al resumir el equilibrio entre *precision* y *recall* de la clase positiva.

Precision IA mide la proporción de predicciones IA que son correctas, mientras que *recall* IA mide la proporción de ejemplos IA detectados. ROC-AUC evalúa la ordenación de los *scores* sin depender de un único umbral [7]. La matriz de confusión muestra la distribución concreta de aciertos y errores entre las dos clases.

La evaluación operacional contempla el tiempo de preprocesamiento, el tiempo de carga del modelo, la latencia de inferencia por fragmento, la latencia de análisis por

canción cuando exista agregación, el tiempo de entrenamiento, el consumo de RAM y VRAM, el tamaño del modelo, la viabilidad de inferencia en CPU y la viabilidad de despliegue en Hugging Face Spaces. Cada experimento documenta únicamente las medidas efectivamente registradas.

4.9. Limitaciones del diseño experimental

El subconjunto experimental contiene 1 000 ejemplos, por lo que solo cubre una parte del material disponible en AIME. Esta reducción permite trabajar con un diseño controlado y equilibrado, aunque limita la variedad de ejemplos que intervienen en la evaluación.

La evaluación se realiza sobre un único corpus. En consecuencia, las medidas obtenidas describen el comportamiento de los modelos dentro de AIME y no proporcionan evidencia directa de generalización a otros conjuntos de datos, generadores o condiciones de audio.

La diferente procedencia de las dos clases introduce otra limitación. La clase humana procede de MTG-Jamendo, mientras que la clase generada reúne ejemplos producidos por varios modelos de IA. Esta configuración puede hacer que parte de la separación aprendida dependa de características de origen y no solo de propiedades generales de la generación musical.

Los metadatos de AIME no incluyen identificadores de canción, artista, álbum o grabación original. En consecuencia, el particionado puede controlar la repetición de combinaciones de etiquetas, pero no establecer grupos basados en la identidad de una obra o de un intérprete.

También existen posibles atajos asociados a diferencias técnicas entre fuentes. La frecuencia de muestreo, el códec, la duración, el proceso de producción o el generador de origen pueden introducir señales que el modelo aprenda sin capturar propiedades generales de la música generada mediante IA. Además, los audios brutos presentan heterogeneidad técnica en duración, frecuencia de muestreo, número de canales, subtipo WAV y tamaño de archivo.

4.10. Conclusiones

La propuesta define un subconjunto equilibrado a partir de AIME, con una muestra humana y una muestra generada por cada grupo semántico. El particionado se realiza de forma reproducible, mantiene balance de clases y evita que una misma combinación de etiquetas aparezca en particiones distintas.

Sobre este diseño se establece el baseline clásico MFCC + SVM y un protocolo común para compararlo con un modelo profundo preentrenado. La comparación se apoya en las mismas particiones, una separación clara entre entrenamiento, validación y prueba, y las medidas predictivas definidas para la evaluación.

5. Validación

5.1. Introducción

Este capítulo presenta la ejecución del baseline clásico MFCC + SVM, la selección de su configuración y los resultados obtenidos. La valoración final se realiza sobre la partición de prueba, acompañada de un análisis de los errores y de las limitaciones específicas del método.

5.2. Entorno y reproducibilidad

La ejecución se realizó con Python 3.11.9 en el entorno virtual del repositorio, donde se instalaron las dependencias del proyecto. Las pruebas automatizadas relacionadas con el baseline se ejecutaron correctamente en este entorno.

El baseline se ejecutó con la semilla 42 y utilizó el manifiesto experimental definido en el capítulo anterior. Las pruebas automatizadas verificaron el preprocesamiento, la extracción de características, la construcción del pipeline, la ausencia de solapamiento entre grupos y la carga del modelo guardado.

Los artefactos principales generados fueron el fichero de características, el modelo entrenado, el fichero de medidas, las predicciones de validación y prueba, y la matriz de confusión de la partición de prueba.

5.3. Ejecución del baseline

La ejecución procesó los 1 000 ejemplos del manifiesto experimental y no registró fallos. Se generaron características para todos los ejemplos y se conservaron las particiones definidas: 700 ejemplos en entrenamiento, 150 en validación y 150 en prueba. Las pruebas ejecutadas confirmaron que los grupos permanecían dentro de una única partición.

5.4. Selección de la configuración

La búsqueda de hiperparámetros aplicó el procedimiento definido en la Sección 4.6 a las 16 configuraciones evaluadas. Cada configuración se entrenó con la partición de entrenamiento y se valoró sobre la partición de validación.

La configuración $C = 10$ y $\gamma = 0,01$ obtuvo en validación una *balanced accuracy* de 0,7800 y un F1 de la clase IA de 0,7871. Este fue el mejor valor de *balanced accuracy*

entre las configuraciones evaluadas, por lo que no fue necesario aplicar el criterio secundario de desempate. Tras la selección, se evaluó en la partición de prueba el modelo entrenado únicamente con la partición de entrenamiento, sin combinar entrenamiento y validación en un nuevo ajuste.

5.5. Resultados predictivos y análisis

La Tabla 5.1 resume las medidas predictivas del baseline. IA se considera la clase positiva; *balanced accuracy* incorpora el recall de ambas clases, mientras que *precision*, *recall* y F1 se muestran para la clase IA. La matriz de confusión permite examinar el comportamiento completo del clasificador. La fila de validación documenta el proceso de selección, mientras que la valoración final del baseline se realiza sobre la partición de prueba.

Partición	Balanced accuracy	Precision IA	Recall IA	F1 IA	ROC-AUC
Validación	0,7800	0,7625	0,8133	0,7871	0,8480
Prueba	0,8333	0,8472	0,8133	0,8299	0,9086

Tabla 5.1: Medidas predictivas del baseline MFCC + SVM en validación y prueba.

En la partición de prueba, el baseline obtiene 125 aciertos sobre 150 ejemplos. El recall humano es $64/75 = 0,8533$ y el recall IA es $61/75 = 0,8133$, de modo que la *balanced accuracy* resulta $(0,8533 + 0,8133)/2 = 0,8333$. La diferencia entre ambos recalls es 0,0400, con una mejor recuperación de la clase humana en esta partición. Esta interpretación se limita al test de AIME utilizado en el protocolo experimental.

La *precision* IA es 0,8472, el recall IA es 0,8133 y el F1 IA es 0,8299. La *precision* IA es ligeramente superior al recall IA, en consonancia con los 14 falsos negativos de la clase IA y los 11 falsos positivos observados en la matriz de confusión.

Como se observa en la Figura 5.1, la matriz de confusión registra 64 ejemplos humanos correctamente clasificados y 61 ejemplos IA correctamente detectados, junto con los errores entre ambas clases.

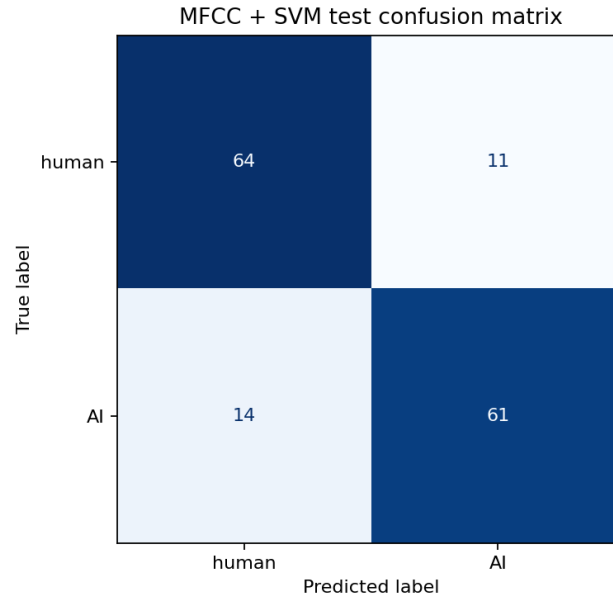


Figura 5.1: Matriz de confusión del baseline MFCC + SVM en la partición de prueba.

El ROC-AUC de prueba es 0,9086, lo que refleja una capacidad favorable de ordenación de los *scores* en el test. Bajo la interpretación habitual de AUC, un ejemplo IA elegido al azar recibe un *score* superior al de un ejemplo humano elegido al azar en aproximadamente el 90,86% de los pares. Los *scores* de *decision_function* no son probabilidades calibradas, y ROC-AUC no depende del único umbral usado por la matriz de confusión. La evaluación mantiene el umbral de decisión del clasificador y no ajusta un umbral adicional mediante la partición de prueba.

5.6. Tiempos de ejecución registrados

La Tabla 5.2 muestra los tiempos registrados por los artefactos del experimento. En ella se recogen exclusivamente las fases temporizadas durante la ejecución del baseline.

Fase registrada	Tiempo
Extracción de características MFCC	2.461,73 s
Entrenamiento y búsqueda de hiperparámetros	0,731 s
Predicción en validación sobre características calculadas	0,012 s
Predicción en prueba sobre características calculadas	0,014 s

Tabla 5.2: Tiempos registrados durante la ejecución del baseline MFCC + SVM.

El tiempo de predicción mide exclusivamente la aplicación de la SVM sobre características previamente extraídas; la extracción de MFCC se registra por separado. Los tiempos disponibles permiten separar la extracción de características del entrenamiento y de la clasificación mediante la SVM.

5.7. Limitaciones del baseline

El baseline utiliza un único fragmento de diez segundos por ejemplo. La selección de la ventana de mayor energía descarta el resto del audio, por lo que la decisión del clasificador depende solo de una parte de cada muestra. Además, la media y la desviación típica de los MFCC eliminan la evolución temporal de los coeficientes.

El modelo no realiza agregación a nivel de canción y puede apoyarse en diferencias acústicas o técnicas presentes en el corpus. Tampoco se ha evaluado en un conjunto externo, por lo que sus resultados deben interpretarse como una referencia experimental dentro del protocolo definido.

5.8. Conclusiones

La ejecución del baseline MFCC + SVM se completó sobre los 1000 ejemplos del manifiesto sin fallos registrados. La configuración seleccionada mediante validación fue $C = 10$ y $\gamma = 0,01$, entrenada sobre la partición de entrenamiento y evaluada finalmente en prueba.

En la partición de prueba, el baseline alcanza una *balanced accuracy* de 0,8333, F1 de la clase IA de 0,8299 y ROC-AUC de 0,9086. Estos resultados constituyen la referencia experimental para la comparación con el modelo profundo preentrenado.

6. Conclusiones

7. Bibliografía

- [1] Darius Afchar, Gabriel Meseguer-Brocal, and Romain Hennequin. 2025. AI-Generated Music Detection and its Challenges. arXiv:2501.10111 [cs.SD]
- [2] Agencia Estatal de Administración Tributaria. 2026. Por coeficientes de amortización lineal. <https://sede.agenciatributaria.gob.es/Sede/ayuda/manuales-s-videos-folletos/manuales-practicos/irpf-2025/c07-rendimientos-actividades-economicas-estimacion-directa/fase-1-determinacion-rendimiento-neto/amortizaciones-dotaciones-ejercicio-fiscalmente-deducibles/requisitos-generales/coeficientes-amortizacion-lineal.html>. Manual práctico de Renta 2025, página actualizada el 17 de marzo de 2026, consultada el 21 de julio de 2026.
- [3] Luca Comanducci, Paolo Bestagini, and Stefano Tubaro. 2024. FakeMusicCaps: a Dataset for Detection and Attribution of Synthetic Music Generated via Text-to-Music Models. arXiv:2409.10684 [cs.SD]
- [4] Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Machine Learning* 20 (1995), 273–297. doi:10.1007/BF00994018
- [5] disco-eth. s. f. AIME: AI Music Evaluation Dataset. <https://huggingface.co/datasets/disco-eth/AIME>. Dataset card de Hugging Face, consultada el 15 de julio de 2026.
- [6] Eurostat. 2026. Electricity prices for household consumers – bi-annual data (from 2007 onwards) (nrg_pc_204). https://ec.europa.eu/eurostat/databrowser/view/nrg_pc_204/default/table. doi:10.2908/NRG_PC_204 Datos actualizados el 19 de mayo de 2026; filtro: España, 2025-S2, EUR/kWh, impuestos incluidos, banda doméstica de 2.500 a 4.999 kWh; consultada el 21 de julio de 2026.
- [7] Tom Fawcett. 2006. An introduction to ROC analysis. *Pattern Recognition Letters* 27, 8 (2006), 861–874. doi:10.1016/j.patrec.2005.10.010
- [8] Florian Grötschla, Ahmet Solak, Luca A. Lanzendörfer, and Roger Wattenhofer. 2025. Benchmarking Music Generation Models and Metrics via Human Preference Studies. arXiv:2506.19085 [cs.SD]
- [9] Chaolei Han, Hongsong Wang, and Jie Gui. 2026. MusicDET: Zero-Shot AI-Generated Music Detection. arXiv:2605.18072 [cs.SD]
- [10] Instituto Nacional de Estadística. 2026. Encuesta Trimestral de Coste Laboral (ETCL). Primer trimestre 2026. <https://ine.es/dyngs/Prensa/ETCL1T26.htm>. Publicada el 16 de junio de 2026, consultada el 21 de julio de 2026.
- [11] Yumin Kim and Seonghyeon Go. 2025. Segment Transformer: AI-Generated Music Detection via Music Structural Analysis. arXiv:2509.08283 [cs.SD]

- [12] Yumin Kim and Seonghyeon Go. 2026. Fusion Segment Transformer: Bi-Directional Attention Guided Fusion Network for AI-Generated Music Detection. arXiv:2601.13647 [cs.SD] doi:[10.48550/arXiv.2601.13647](https://doi.org/10.48550/arXiv.2601.13647)
- [13] David Lopez-Ayala, Asier Cabello, Pablo Zinemanas, Emilio Molina, and Martin Rocamora. 2026. AI-Generated Music Detection in Broadcast Monitoring. arXiv:2602.06823 [cs.SD]
- [14] Zain Nasrullah and Yue Zhao. 2019. Music Artist Classification with Convolutional Recurrent Neural Networks. arXiv:1901.04555 [cs.SD]
- [15] Sergey Novoselov, Alexandr Kozlov, Galina Lavrentyeva, Konstantin Simonchik, and Vadim Shchemelinin. 2015. STC Anti-spoofing Systems for the ASVspoof 2015 Challenge. arXiv:1507.08074 [cs.SD]
- [16] Heewon Oh. 2026. ArtifactNet: Detecting AI-Generated Music via Forensic Residual Physics. arXiv:2604.16254 [cs.SD]
- [17] Md Awsafur Rahman, Zaber Ibn Abdul Hakim, Najibul Haque Sarker, Bishmoy Paul, and Shaikh Anowarul Fattah. 2024. SONICS: Synthetic Or Not – Identifying Counterfeit Songs. arXiv:2408.14080 [cs.SD]
- [18] Yongyi Zang, Jiatong Shi, You Zhang, Ryuichi Yamamoto, Jionghao Han, Yuxun Tang, Shengyuan Xu, Wenxiao Zhao, Jing Guo, Tomoki Toda, and Zhiyao Duan. 2024. CtrSVDD: A Benchmark Dataset and Baseline Analysis for Controlled Singing Voice Deepfake Detection. arXiv:2406.02438 [cs.SD]
- [19] Yongyi Zang, You Zhang, Mojtaba Heydari, and Zhiyao Duan. 2023. SingFake: Singing Voice Deepfake Detection. arXiv:2309.07525 [cs.SD]